

JavaScript Fundamentals: The Developer's Toolkit

Unlocking the 5 Essential Function Types



**Named
Functions**



Expressions



**Arrow
Functions**



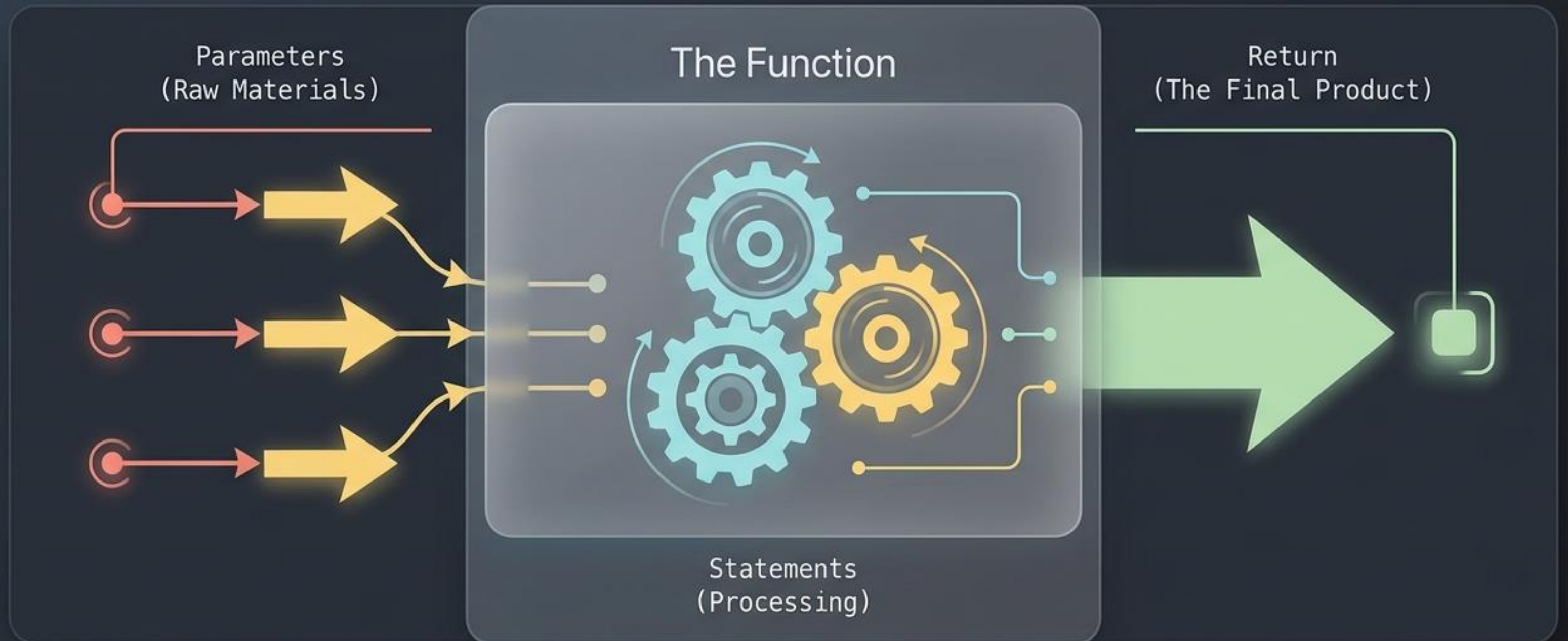
IIFE



Callbacks

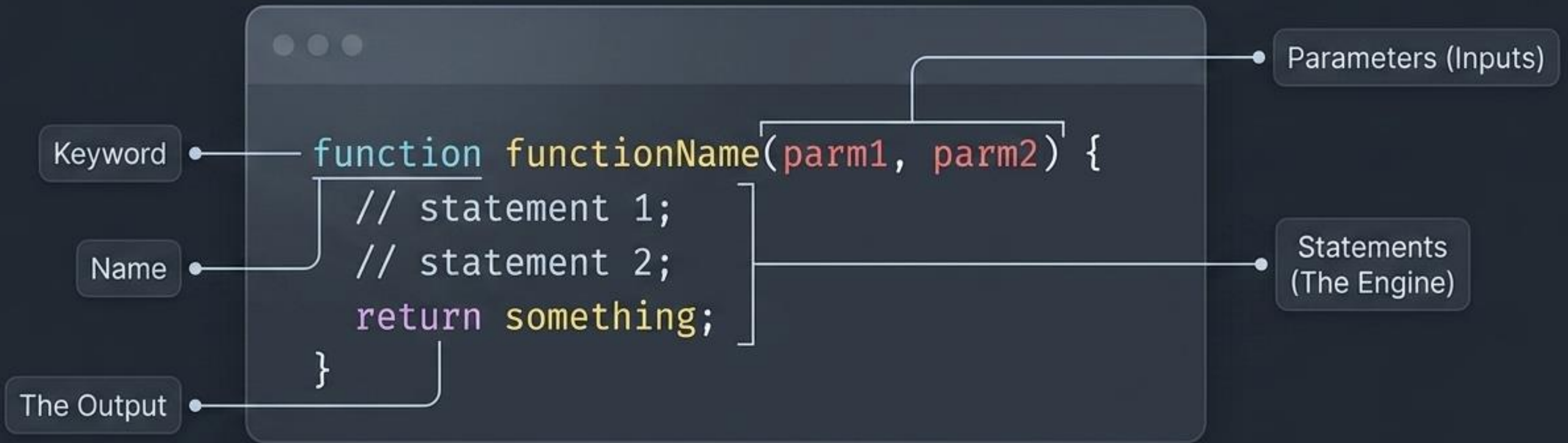
What is a Function?

Think of it as a blueprint for a processing machine.



1. Named Function Declarations

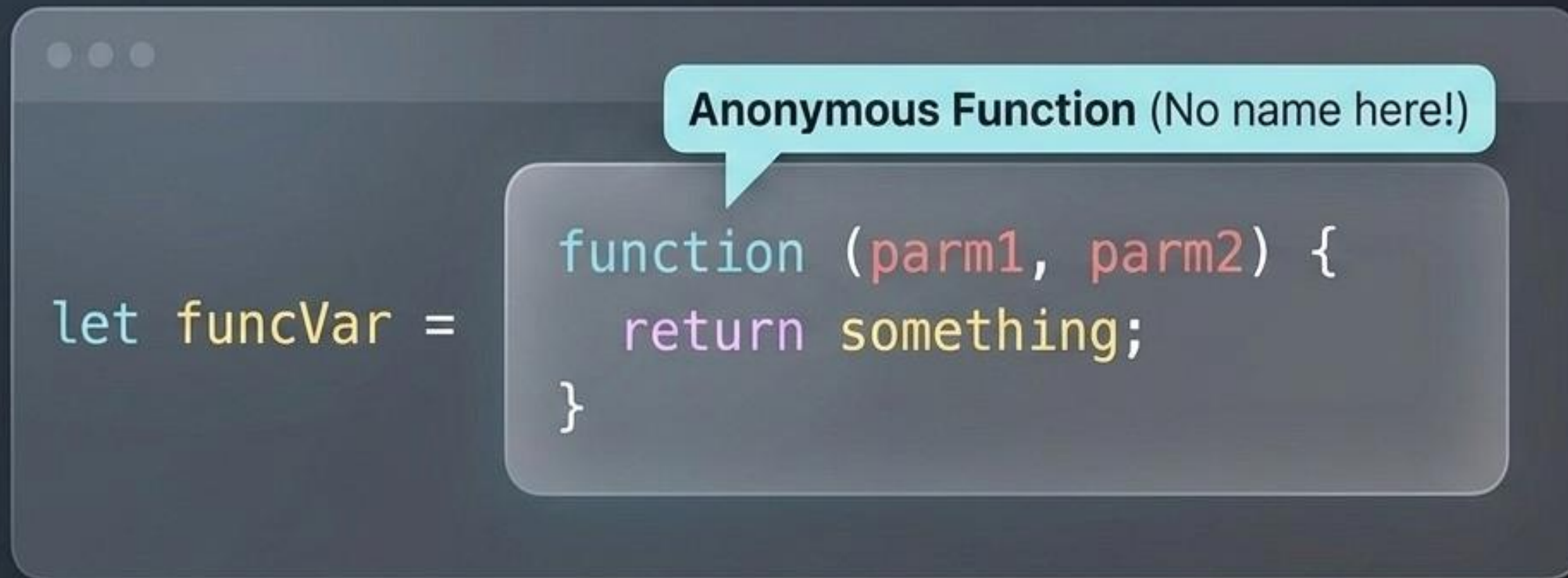
The standard, reusable tool.



Calling the function: `functionName(parm1, parm2);`

2. Function Expressions

Storing a nameless machine inside a variable.



3. Arrow Functions

The modern, shorthand tool.

```
let funcVar =  
function(parm1) {  
  ...  
}
```

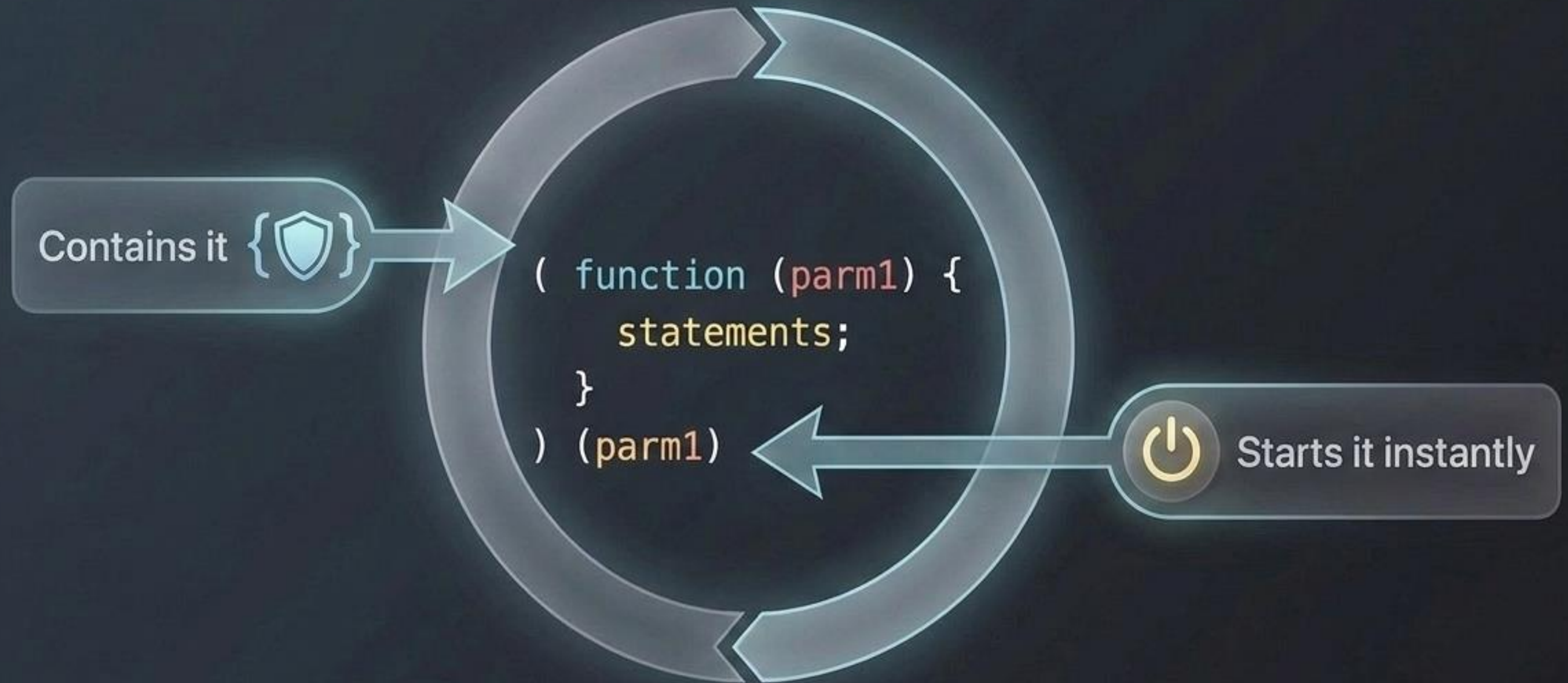
Replaced with the
FAT ARROW

```
let funcVar =  
  (parm1) => {  
    ...  
  }
```

Pro-Tip: `const add = (a, b) => a + b; // simple & concise`

4. Immediately Invoked Function Expression (IIFE)

The self-starting engine. Runs as soon as it is defined.



5. The Callback Concept

Passing the baton to another function.



The value of a variable is passed using a callback function as an argument of another function.

Callbacks in the Real World

Handling asynchronous delays.



The Diagnostic Matrix

When to use which tool in your kit.

Tool (Function Type)	Hoisted?	Best Use Case
Named Function	 Yes	Reusable functions that should be available globally
Function Expression	 No	Assigning functions to variables, limiting scope
Arrow Function	 No	Shorter syntax, useful for callbacks
IIFE (Immediately Invoked Function Expression)	 No	Running code once, avoiding global variable pollution

Key Takeaway

Why do we use functions?

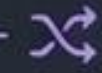
Write Once,
Use Anywhere



Keep Logic
Organized



Pass Actions
as Arguments



**Functions allow us to write
reusable, modular code.**